applied sciences

MDPI

Article

Service Reliability Based on Fault Prediction and Container Migration in Edge Computing

Lizhao Liu [1], Longyu Kang [1], Xiaocui Li [1] and Zhangbing Zhou [1,2,*]

[1] School of Information Engineering, China University of Geosciences (Beijing), Beijing 100083, China; 13908065625@163.com (L.L.); fangke_860@sina.com (L.K.); lixiaocui.cugb@gmail.com (X.L.)
[2] Computer Science Department, TELECOM SudParis, 91000 Evry, France
* Correspondence: zbzhou@cugb.edu.cn

Abstract: With improvements in the computing capability of edge devices and the emergence of edge computing, an increasing number of services are being deployed on the edge side, and container-based virtualization is used to deploy services to improve resource utilization. This has led to challenges in reliability because services deployed on edge nodes are pruned owing to hardware failures and a lack of technical support. To solve this reliability problem, we propose a solution based on fault prediction combined with container migration to address the service failure problem caused by node failure. This approach comprises two major steps: fault prediction and container migration. Fault prediction collects the log of services on edge nodes and uses these data to conduct time-sequence modeling. Machine-learning algorithms are chosen to predict faults on the edge. Container migration is modeled as an optimization problem. A migration node selection approach based on a genetic algorithm is proposed to determine the most suitable migration target to migrate container services on the device and ensure the reliability of the services. Simulation results show that the proposed approach can effectively predict device faults and migrate services based on the optimal container migration strategy to avoid service failures deployed on edge devices and ensure service reliability.

Keywords: fault prediction; container migration; service reliability; edge computing

Citation: Liu, L.; Kang, L.; Li, X.; Zhou, Z. Service Reliability Based on Fault Prediction and Container Migration in Edge Computing. *Appl. Sci.* **2023**, *13*, 12865. https://doi.org/10.3390/app132312865

Academic Editor: Gerard Ghibaudo

Received: 27 September 2023
Revised: 28 November 2023
Accepted: 29 November 2023
Published: 30 November 2023

1. Introduction

In recent years, edge computing has become a prominent concept in cutting-edge research and technology. Edge nodes are closer to the terminal devices of users and hence speed up data processing and transmission. This is very important for real-time applications that are highly sensitive to low latency, such as face recognition using artificial intelligence (AI) algorithms. Users can benefit from edge services by offloading their computing tasks to nearby edge servers. Thus, users obtain higher bandwidth, lower latency, and more computing power. This results in a better user quality of service (QoS) and user experience, which are important for many AI and Internet of Things (IoT) applications.

Several recent studies on edge computing have focused on energy consumption, low latency, and service security to improve the QoS. Jalali et al. [1] studied fog calculations in an energy-consuming environment and compared them with energy consumption in cloud centers. They proved that edge computing can reduce the energy consumption of the system. Hu et al. [2] studied the deployment of services in a mobile edge environment to reduce the average response time of edge servers and improve the QoS for users, while Zhao et al. [3] used experiments in Wi-Fi and LET networks to demonstrate that edge computing could significantly improve the response time and reduce the energy consumption of mobile devices. Wu et al. [4] proposed an aggregation encryption scheme to solve the problem of data transmission security in an edge-computing environment. This scheme reduces the amount of computation and communication overhead and ensures the

security and effective provision of edge computing. These studies mainly aimed to improve resource utilization and reduce service latency for edge computing. However, related studies on service reliability in edge-computing environments are limited. In practice, edge computing is more vulnerable to hardware failure than traditional cloud computing owing to geographical decentralization and a lack of technical support. Device/node failure is one of the main reasons for a service shutdown [5]. Therefore, in an edge environment, it is necessary to take measures to ensure service reliability and avoid service failures caused by device downtime.

In cloud computing, service reliability can be improved through fault prediction [6]. By collecting data on the status of the equipment over a long period, we can build a model based on historical data. Hence, by monitoring the real-time status of the equipment, we can forecast its future status based on an information model built using historical data. Thus, the cloud-computing environment can be monitored in real time. This approach works well in cloud-computing services, but in an edge environment, it is not very effective when applied in a straightforward manner. In contrast to cloud-computing center devices, it is not feasible to have uninterrupted human operation and maintenance for edge devices owing to their geographically scattered locations. Even if only equipment failure predictions are made, equipment downtime cannot be reduced and, therefore, service failure cannot be avoided. Therefore, an automatic fault-tolerance mechanism must be applied to ensure service reliability.

With the development of container technology, the use of containers for deploying edge services has become important. A container is a relatively independent environment similar to that of a virtual machine. Containerization separates the application configuration from the underlying hardware and enables multiple applications to share the same operating system on a single physical machine, thereby significantly reducing the cost of server operation and maintenance. Owing to their lightweight nature, container technologies can be combined with edge computing. Therefore, services in edge environments are being gradually provided in the form of containers. Moreover, real-time container migration is an important technology [7] that has triggered extensive research by technicians. Through real-time container migration, the containers in one device can be migrated to another device without user awareness. Generally, container transfers can be completed within 30 s. These values can reach millions of seconds under various optimization methods and conditions. Therefore, container migration can be effectively applied to solve reliability problems using a similar approach to fault recovery or load balancing.

The major contribution of this paper is that we propose a novel two-phase method, the combination of fault prediction and container migration, on the edge computing. The fault prediction is used to decide when to trigger the container migration. The migration is to prevent the service failure. We first forecast a container failure and then migrate the container to another healthy node. We develop several approaches for both steps and select the most suitable approach to address this issue. In Section 3, we describe problems with the system model. In Section 4, we introduce fault prediction and cost optimization for migration. Machine-learning algorithms are applied to historical data to build a model for fault prediction. Based on this fault prediction, the container migration strategy is modeled as an optimization problem, and a migration node selection algorithm based on a genetic algorithm (GA) is proposed to determine the appropriate migration target in healthy edge devices and migrate the container services on the devices. Using the hot-migration technology of containers, services are migrated to other devices before a device failure occurs. The migration cost is reduced as much as possible to balance the load on the edge network, which simultaneously increases its stability. Section 5 describes simulation experiments conducted using the proposed methods, including the prediction of equipment failures based on historical datasets and the effectiveness of the container migration strategy based on failure prediction. The experimental results demonstrate that the proposed method can effectively predict faults, reduce migration costs, and ensure the load balance of the edge network to ensure the reliability of services.

2. Related Work

2.1. Task Offloading in Edge Computing

In recent years, the detection and prediction of anomalies and failures have aroused significant interest in the research community. These methods can be classified into detection and prediction schemes. Detection schemes identify faults at the time of failure, whereas prediction schemes attempt to predict faults before they occur. Fault prediction is used to predict the operating status of the equipment using relevant prediction methods based on current equipment information [8]. According to Tellme Networks, for system recovery in a distributed environment, 75% of the recovery time is used to detect system failures, and 18% is used to diagnose failures [9]. In general, 65% of failures can be avoided if failure detection is executed on the face [10]. Therefore, efficient and accurate detection of edge server failures is key to service reliability.

Equipment failure prediction can be primarily classified into prediction methods based on mechanism models and data-driven prediction methods. Wang et al. [11] proposed a fault detection method for cloud-computing systems based on adaptive monitoring. By monitoring various attributes of the system, a correlation analysis was conducted to establish the correlation between each measure, and key measures were selected for system monitoring. Principal component analysis (PCA) was used to analyze the data. In this manner, principal feature vectors were extracted to describe the operating state of the system and hence establish a reliability model for the system that can be used to predict system failure.

For some devices and systems, owing to their complex internal structures or model parameters, it is impossible to directly use mathematical or physical methods to build a mechanism model. Data-driven fault-prediction methods have been developed for such cases. In data-driven fault prediction, the future state performance of the equipment is predicted using historical data on the operating status of the equipment and relevant models to obtain the final fault-prediction results effectively and efficiently [12]. In recent years, with the continuous development of machine-learning technology, an increasing number of machine-learning algorithms have been used in data-driven prediction methods because they can establish the correlation between data and failure states without accurate mechanism models in advance.

Data-driven fault-prediction technology is based on real data and extracts the implied information for prediction through data analysis and processing methods. This has become a practical fault-prediction technology. In [13], an online detection model called a support vector machine (SVM) grid, which is based on an SVM, was used to predict the cloud status. This approach applies PCA to select monitoring parameters and reduce dimensionality. It also optimizes the SVM parameters using a grid network. Thus, the faults in a cloud system can be detected and predicted efficiently.

In [14], a deep neural network (DNN) based on Gaussian Bernoulli restricted Boltzmann machines (GBRBM) was proposed for IoT devices under the Industrial Internet of Things. The algorithm transforms the fault detection problem into a classification problem to achieve accurate fault detection.

Although there are many studies on fault prediction in cloud environments, few have focused on fault prediction in edge-computing environments, where modeling and fault prediction must be performed according to the characteristics of the edge environment.

2.2. Service Migration in Edge Computing

Currently, several studies focusing on container migration strategies have been reported, and most of these are in mobile edge-computing environments or consider load balancing of the edge network. The purpose of a container migration strategy is to move containers from overloaded nodes to relatively healthy nodes with a high migration efficiency when container migration is triggered. In summary, a container migration strategy addresses two main issues: container selection for migration and container remapping.

In [15], the author proposed a minimum transfer time selection strategy to minimize container transfer to end users. In this study, all containers for migration were selected using overloaded node sorting. The migration time was calculated as the ratio of the memory resources used by the container to the bandwidth available to the device. The container with the shortest migration time was selected to minimize the migration time.

On this basis, in [16], the authors not only considered the migration time but also comprehensively considered the energy consumption of physical and virtual machines. This approach migrates the virtual machines with the lowest CPU utilization on overloaded physical machines and realizes the selection strategy of maximum migration benefit by combining the migration time and energy cost.

In [17], the authors first considered the prediction of the future load of nodes and then considered the migration quantity, energy use, and communication overhead between virtual machines to achieve the optimal migration strategy in the case of multiple objectives. In [18], to avoid service quality degradation caused by resource overload on physical machines, the authors proposed a migration selection strategy for maximum CPU utilization. Beloglazov et al. [19] proposed the random container migration strategy (RCMS), which is a virtual machine selection algorithm based on a minimum migration strategy. It randomly generates uniform discrete values and then determines the devices to be migrated according to these values. In [20], a service migration mechanism based on mobile awareness was proposed to solve the problem of multinode service migration in smart contract edge computing. By optimizing business selection and finding the corresponding destination nodes, delays in business and service can be reduced, and the QoS can be improved. However, in these studies, the migration timing considered in the load-balancing solution using the container migration strategy was mostly for edge node overload. Omar et al. (2023) applied LSTM and GPT2 to classify software code for potential vulnerabilities [21]. Topaloglu et al. (2023) used machine-learning algorithms on the price forecasting on e-commerce data [22]. To address this gap in the literature, in this study, the container migration timing is determined by fault prediction in the first step.

Container migration is a typical non-deterministic polynomial (NP)-hardening problem. Most existing studies have used heuristic algorithms to obtain high-quality approximate solutions. Wang et al. [23] proposed a dynamic virtual machine migration strategy. Their virtual machine selection strategy was based on CPU utilization. The virtual machine with the highest priority migrates to the virtual machine that can maximize the load-balance degree. The procedure was repeated until all the virtual machines selected for migration were complete. Wen et al. (2017) proposed a container migration algorithm based on an improved GA [24]. The container migration problem was solved effectively using the encoding, crossover, and mutation operations of a GA. In [25], a multi-objective ant-colony algorithm was proposed, which comprehensively considered the total resource waste and power consumption required to effectively save energy. Li et al. (2021) proposed a decision mechanism and migration algorithm based on container migration [26]. By improving the ant-colony algorithm and establishing a local pheromone volatilization model and a global pheromone update model, the optimal migration of containers can be realized, the load balance can be improved, and the migration cost can be reduced. Xin et al. (2020) proposed a container migration mechanism based on load balancing [27]. First, the timing of the container migration was determined based on the classical static resource utilization threshold model. Subsequently, a container migration model of load-balancing combined migration cost was established to minimize the impact of container migration while balancing the load of the edge network. The container migration mechanism proposed in this paper not only improves the load balancing of the edge network but also reduces the cost of container migration.

However, these studies were based on the container migration caused by node overload. Most studies have only considered the load balancing of the network after migration, without considering the migration costs generated in the migration process. Additionally, no studies have considered a model in which migration timing is first determined by the

fault prediction of devices in the edge environment and then the overall load balance of the edge network after migration and the migration cost caused by the migration of containers from the failed device to the healthy device are comprehensively considered. Therefore, this study proposes a migration strategy based on a GA, which considers fault prediction as the judgment condition of migration timing and minimizes the migration cost and load balancing of edge networks after migration.

3. System Model

In this section, we introduce the system model, which contains three parts: network, migration cost, and load-balancing models. These models define the problems that must be solved.

3.1. Network Model

Definition 1. *Edge Node: An edge device is a tuple EdgeN = (Id, Loc, F, EC, EM, ES), where:*

1. *Id represents the unique identifier of the edge device;*
2. *Loc represents the geographic location of the edge device, which includes the longitude and latitude;*
3. *F represents the state of the edge device, which is divided into normal and fault states;*
4. *EC represents the CPU capacity of the edge device;*
5. *EM represents the memory capacity of an edge device;*
6. *ES represents the storage capacity of the edge device.*

Definition 2. *Container Service: A container service is a tuple Caas = (Id, CC, CM, CS, CD), in which*

1. *Id represents the unique identifier of the container service;*
2. *CC indicates the size of CPU resources required by the container;*
3. *CM indicates the size of memory resources required by the container;*
4. *CS represents the size of storage resources required by the container;*
5. *CD represents the size of data resources transmitted by the container.*

Definition 3. *Edge Network: An edge network is a tuple EN = (DN, CN, FEN), where*

1. *DN represents all the devices in the network;*
2. *CN represents all the containers in the network;*
3. *FEN represents all the predicted failure devices in the network.*

In this edge network architecture, edge devices are evenly distributed in the network, and services are provided in the form of containers, which can be deployed simultaneously on the same device. Each container can only host one service in it; there is a predicted failure group of edge devices in the network, and the containers running on this group of devices will be migrated to the remaining healthy devices in the form of live migration.

3.2. Migration Cost Model

During the container migration process, the container migration cost mainly consists of two parts: the network delay caused by the transmission of intermediate and final result data between any edge nodes and the migration time of the container itself [8], that is, the downtime. The downtime is mainly affected by the container memory size [28]; therefore, this study uses the following equation to calculate the migration cost incurred when container *Ci* migrates from edge node N_i to N_j.

$$Mig^{i}_{cos\ t} = \frac{CD_i + CM_i}{Band_{i,j}} \tag{1}$$

where CD_i represents the amount of data transmitted by container C_i; CM_i represents the memory resources allocated to container C_i; and $Band_{i,j}$ represents the available network bandwidth between devices i and j. $Mig^i_{cos\ t}$ represents the cost of migrating the deployed container to another healthy device. Therefore, for all the faulty devices in the edge network, the total migration cost is given by the following:

$$Mig^{total}_{cos\ t} = \sum_{c_i \in C} y_{i,j} Mig^i_{cos\ t} \tag{2}$$

Here, C represents all containers on the faulty device that need to be migrated, and $y_{i,j}$ is a binary variable, which represents whether or not there exists migration from faulty device i to healthy device j based on Equation (1).

3.3. Load-Balancing Model

After migrating the containers on all faulty devices to healthy devices, we need to evaluate the overall load-balancing degree of the network from two aspects: (1) the resource utilization balance of the same type of resources among edge devices as defined in Definition 1, which is denoted by U, and (2) the resource balance of different types of resources on the edge device, denoted by V. The load-balancing formula is as follows:

$$Load_{balance} = U^{total}_{balance} + V^{total}_{balance} \tag{3}$$

Let R denote the type set of all resources on the device, expressed as $R = \{r_1, r_2, \ldots r_n\}$. For any resource type r_k, the resource utilization of r_k in the edge network occurs after the container on the faulty device migrates. The equilibrium degree is the variance of the resource utilization of r_k on all edge devices. The method of calculating the equilibrium degree for a certain type of resource utilization is as follows:

$$U^k_{balance} = \sum_{n_j \in J} \frac{(u^k_j - \overline{u})^2}{J} \tag{4}$$

where u^k_j represents the resource utilization of resource type r_k on the device after the container is migrated to edge device n_j. $\overline{U^K}$ represents the average resource utilization of resource type r_k on all edge devices. J represents the current network. Based on this formula, we can evaluate whether the utilization of a certain type of resource r_k is balanced among the edge devices.

Using Equation (4), the total resource utilization equilibrium degree in the edge network is the sum of the resource utilization equilibrium degrees of all types of resources r_k. This is expressed as follows:

$$U^{total}_{balance} = \sum_{r_k \in R} U^k_{balance} = \sum_{r_k \in R} \sum_{n_j \in J} \frac{(u^k_j - \overline{u})^2}{J} \tag{5}$$

For any edge device $n_j \in$ N, the remaining resource vector is defined as $\overline{S_j} = \left(s^1_j, s^2_j, \ldots, s^k_j\right)$, where s^k_j represents the remaining available resources of resource type r_k on device n_j after container migration is completed from the faulty device to the healthy device. This is defined as follows:

$$S^k_j = W^k_j - \sum_{c_i \in C} d^k_i \tag{6}$$

Here, W^k_j represents the total resource amount of resource type r_k on edge device n_j, and d^k_i represents the resource amount of resource type r_k that container c_i needs to occupy.

Therefore, the remaining resource equalization degree of edge device n_j can be expressed as follows:

$$V_{balance}^{j} = \sum_{\forall r_{k_1}, r_{k_2} \in R} max\left\{0, \frac{S_j^{k_1}}{W_j^{k_1}} - \frac{S_j^{k_2}}{W_j^{k_2}}\right\} \quad (7)$$

Here, $\frac{s_j^{k_1}}{w_j^{k_1}}$ represents the proportion of the remaining available k_1-th resources on n_j to the total number of resources on the k_1-th resource. This formula reflects whether the utilization of the various resource types in n_j is balanced.

Therefore, the total remaining resource balance degree of the edge network is the sum of the remaining resource balance degrees for all edge devices. The calculation method is shown as follows:

$$V_{balance}^{total} = \sum_{n_j \in J} V_{balance}^{j} = \sum_{n_j \in J} \sum_{r_1, r_2 \in R} max\left\{0, \frac{S_j^{k_1}}{W_j^{k_1}} - \frac{S_j^{k_2}}{W_j^{k_2}}\right\} \quad (8)$$

It can be seen from the above definition that the smaller the values of the resource utilization balance degree and the remaining resource balance degree, the more balanced the load of each device in the network.

3.4. Problem Formulation

The container migration optimization objective for edge network load balancing is defined as the weighted sum of all the above influencing factors and is expressed as follows:

$$F = \theta\left(U_{balance}^{total} + V_{balance}^{total}\right) + \gamma Mig_{cos\ t}^{total} \quad (9)$$

Here, θ and γ are the weighting factors of the load-balancing degree of the edge network and the container migration cost, respectively, where θ and γ are in the range of [0, 1] and $\theta + \gamma = 1$.

4. Failure Prediction and Container Migration Strategies

4.1. Problem Formulation

Fault-prediction models are very important for the fault prediction of edge devices. High-precision fault prediction can reduce unnecessary migration costs caused by incorrect judgments, which is significant for subsequent migration strategies. There are several technical challenges in designing failure prediction models for edge devices. First, edge-device failures can be caused by software or hardware issues. According to estimates by Microsoft domain experts, more than a hundred root causes of edge-device failures exist. Examples include operating system crashes, application errors, disk failures, misconfigurations, memory leaks, software incompatibilities, overheating, and service exceptions. In such complicated scenarios, simple rule- or threshold-based models cannot achieve accurate predictions. Second, there is a high imbalance in the percentage of failure data. In general, the equipment failure data is considerably less than normal data. This imbalance makes it difficult to train the prediction model. Hence, data must be preprocessed to balance normal and failure data. To address these two challenges, we introduce a machine learning-based approach for edge-device failure prediction. Data preprocessing is described in Section 5.

Through long-term monitoring and analysis of various performance indicators of the nodes, changes in the node performance indicators under normal operation were found to be regular and stable. When a node fails, its performance index can fluctuate abnormally either suddenly or gradually, which is a symptom of node failure [29]. Therefore, it can be concluded that the node indicators are correlated in the time dimension. In addition, although the failure types are different, the nodes often experience a period of state evolution between failure states. In other words, when the performance parameters of a certain

time node are compared with those of others, a significant difference indicates that a failure is likely to occur. The specificity, variation, and correlation of such node performance parameters in the time dimension may be key to early symptom identification of node failures. Through these comprehensive considerations, this study considers establishing the fault prediction of edge devices as a time-series model and can use some typical machine-learning algorithms related to time-series prediction to predict the fault situation at a certain moment in the future using the data from the previous period. Specifically, this study used two well-known time-series machine-learning algorithms: long short-term memory (LSTM) and gated recurrent unit (GRU) algorithms.

4.1.1. LSTM

The LSTM algorithm, first proposed by Hochreiter et al. in 1997 [30], is a temporal recurrent neural network (RNN) suitable for processing and predicting important events with relatively long intervals and delays in a time series. We applied it to failure prediction as described in Algorithm 1. LSTM assumes that RNNs can have a long memory of weights and that the weights change slowly during training. The LSTM model introduces an intermediate key with a memory function, called a memory cell. In contrast to an RNN, each neuron in the hidden layer of an LSTM is replaced by a memory cell. As shown in Figure 1, each storage cell has a composite structure with simple nodes. A standard LSTM memory cell should contain the following components:

- Input node g_c: The state of the input data point X_1 of the current time step and the output h_{t-1} of the previous time step are weighted using the *tanh* activation function, which is primarily used to update the current state of the LSTM cells and add new information for the current state.
- Input gate i_c: The input gate i_c weighs X_t and h_{t-1} using a sigmoid activation function. It is called a "gate" because the value produced by passing through it is multiplied by the value of another node. If its value is zero, the traffic from another node is eliminated. If its value is 1, all information from the other node passes.
- The cell state S_c: The S_c is the central part of the LSTM memory cell, which has a self-connecting circular edge that functions as a cellular memory. When the cell state is transferred in adjacent time steps, this edge has its own fixed weight so that the gradient does not disappear or explode.

Algorithm 1 Device failure prediction based on LSTM.

```
Input: Trained LSTM model, a set of time-series data from edge devices
Output: prediction result for the next time slot
Procedure:
1: while (there exists new measurement data x)
2: do
3:        result = predict (LSTM model, data x) # predict the next step;
4:        if the result shows failure
5:          output the result and break;
6: end
```

4.1.2. GRU

A GRU is a temporal RNN model that can remember the influence of historical input information on subsequent output results, as well as specific information at a particular time. It is very suitable for processing time-series data and extracting relevant information about adjacent features in the time dimension [31]. As shown in Figure 2, the large-capacity deep network obtained through the layer-by-layer connection of GRU neurons can be used to process complex sequence data, further improving prediction accuracy. Therefore, considering the complexity of fault symptoms, we applied a multilayer GRU deep network to extract the temporal characteristics in the time window before failure, as shown in Algorithm 2.

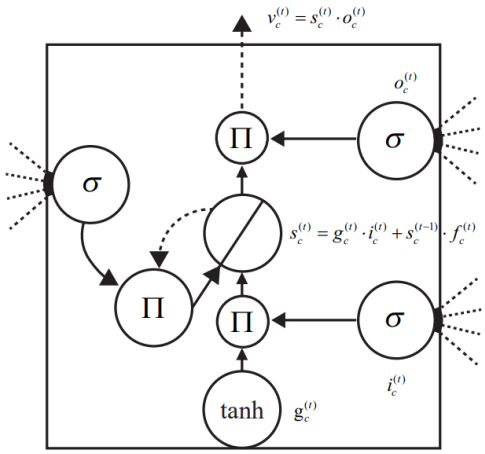

Figure 1. Standard LSTM neuron structure.

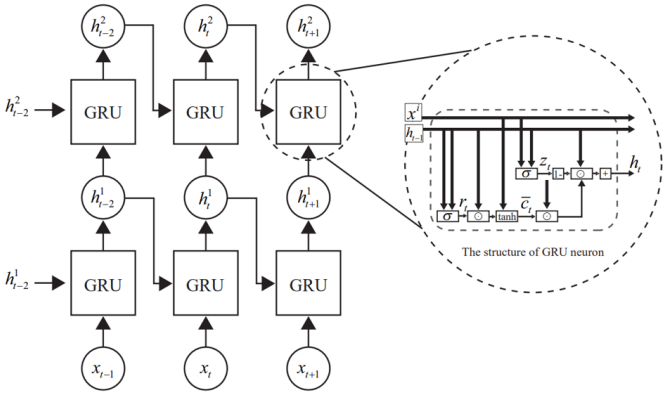

Figure 2. Two-layer GRU model.

However, multilayer GRU deep networks have certain drawbacks. First, owing to a high level of abstraction, some features that are important for distinguishing the operational state of a node and the type of failure are lost, contrary to our initial goal of obtaining more feature details. Second, with the deepening of the GRU network, the training efficiency of the DNN is reduced, which is not suitable for online failure prediction. Finally, the tanh activation function is prone to gradient vanishing problems, resulting in wandering at a certain point and an inability to determine the optimal solution.

Algorithm 2 GRU-based fault-prediction algorithm.

Input: Trained GRU model: A set of time-sequence data from edge devices.
Output: Edge-device failure prediction results at the next moment
Procedure:
1: while (there exists new measurement data x)
2: do
3: result = use the GRU model to predict the next failure situation;
4: if the result is failure
5: output the result and break;
6: end while

4.2. Container Migration Strategies

Let N denote the total number of devices in an edge network. If K ($K < N$) devices are predicted to be faulty, the encoding method for each chromosome in the GA-based migration strategy is as follows:

$$Chromosome = [n_i, n_j, \ldots, n_k]_k \tag{10}$$

Among them, $n_i, n_j, \ldots, n_k$ represent the unique identification ID of the healthy device, and each device number corresponds to the container on the faulty device one-to-one. In other words, the container on the faulty device is migrated to the device with the corresponding serial number.

According to the coding method, it is necessary to use an evaluation function to calculate the fitness of each individual using the following formula:

$$fitness = \frac{1}{\left(\theta\left(U_{balance}^{total} + V_{balance}^{total}\right) + \gamma Mig_{cos\ t}^{total}\right)} \tag{11}$$

Subsequently, according to the fitness calculated in the previous step, the classic roulette algorithm is used to screen and select excellent individuals. The following formula is used:

$$P_i = \frac{f_i}{\sum_{i=1}^{NP} f_i} \tag{12}$$

In this equation, f_i indicates the fitness of an individual and NP is the total number of individuals.

The selected individuals avoid falling into a local optimal solution through single-point crossover and single-point mutation. Finally, when the number of iterations reaches a preset number, the algorithm terminates, and the individual with the largest fitness function value is output as the optimal migration strategy.

Algorithm 3 describes the container migration strategy algorithm based on a GA. After initializing the population, for each generation of individuals, it first calculates the fitness value (lines 5–9) according to Equation (11), and then performs selection, crossover, and genetic operations (lines 10–18). After the number of iterations is reached, the output is a heuristic migration strategy.

Algorithm 3 Migration strategy based on a GA.

Input:
EN: Edge network deployment, including container list and device list
FEN: A list of faulty nodes, including all predicted faulty devices in the current time slice
Output:
A_t: optimal migration strategy
Procedure:
1: Initialize population List P = {p_1, p_2, …, p_s}
2: Initial evolutionary parameter t = 0, the maximum number of iterations T
3: while t < T do
4: s = len(p)
5: for i = 1 to s do
6: MigCost ← calMigCost(Fen)
7: loadBalance ← calLoadBalance(EN - Fen)
8: fitness = 1/(γMigCost + θloadBalance)
9: end for
10: for i = 1 to S/2 do
11: crossover operation to p(i)
12: end for
13: for i = 1 to S do
14: variation operation to p(i)
15: end for
16: for i = 1 to S do
17: P(i + 1) = P(i)
18: end for
19: t = t + 1
20: end while
21: output(MS)

5. Performance Evaluation

Our experimental environment was constructed on a 64-bit Windows 10 operating system with 16 GB of memory, an Intel (R) Core (TM) i7-3770 CPU @ 3.40 GHz, and NVIDIA GeForce GTX 1070 Ti. The algorithms were developed using Python software. All experiments were conducted in the same operating environment.

5.1. Experimental Setup

The network size was simulated as 500 m × 500 m. Edge devices were evenly deployed in the geographic area, and only one container was deployed on each edge device. The edge devices currently considered are all homogeneous; that is, the resources (CPU, memory, and storage size) of each device are identical. Simultaneously, the edge devices can communicate with each other, and container migration can be performed between any pair of edge devices. Different containers require different amounts of resources.

For preprocessing data for failure prediction, we used the public dataset BigML, which contains 8784 machine data with a recording interval of one hour, where each piece of data contains 28 attributes. Simultaneously, to accurately predict failure and provide sufficient time for subsequent container migration, the average interpolation method was used to interpolate the data at one hour intervals. That is, the average value of two consecutive records was calculated, and this calculated value was inserted between the two consecutive records. Finally, the interval between every two records was set to 15 min. Thus, we used data from the first 45 min to predict the state of the subsequent 15 min. The BigML dataset is a label-imbalanced dataset, with 8703 data points labeled as non-failure and 81 data points labeled as failure. We adopted the SMOTE algorithm [32] to solve this data imbalance problem, thereby ensuring the effectiveness of fault prediction.

5.2. Experimental Results and Evaluation Analysis

This experiment first explored two typical time-series machine algorithm models, LSTM and GRU, and then compared their precision and recall. Fault-based container migration strategies mainly explore the scale of the edge network and the number of faulty devices for container migration. We evaluated the migration cost and the impact of load balancing on the process. Similarly, Beloglazov et al. [19] compared their proposed Genetic Algorithm Container Migration Strategy (GACMS) with the Greedy Container Migration Strategy (GCMS) and RCMS.

5.2.1. Failure Prediction Accuracy

A fault-prediction model was used to predict the fault conditions of edge devices at the edge end. A fault-prediction model with high precision can effectively avoid container migration caused by misjudgment, thus avoiding large migration costs and load-balance fluctuations caused by invalid migration. Therefore, the analysis shows that the accuracy of the fault-prediction model has a significant impact on the determination of the subsequent migration strategies. The higher the accuracy of the fault-prediction model, the more the service reliability will be improved in the edge environment, and the higher the possibility of avoiding service failure. In our experiments, accuracy is the percentage of correctly predicted faulty nodes compared to all predicted faulty nodes. Recall is the percentage of correctly identified faulty nodes compared to all actual faulty nodes.

Figure 3 shows the experimental results of LSTM and GRU. As shown in the figure, the GRU model performed better than the LSTM model in terms of both precision and recall. The GRU model has a precision of 0.94 and a recall of 0.85. Analysis of the experimental results suggests that this difference is because the dataset used in this study was small, and the GRU model is a simplification of the LSTM model, which may perform better when there is less data. Considering the historical data collection of edge devices in practical situations, it is challenging to collect a large amount of data for training under normal circumstances. Therefore, using the GRU model, which performed better on the dataset used in this study for the fault prediction of edge devices, can avoid unnecessary migration

costs caused by prediction failures when both the precision and recall rates are relatively high. If the prediction is successful, service reliability can be guaranteed through container migration.

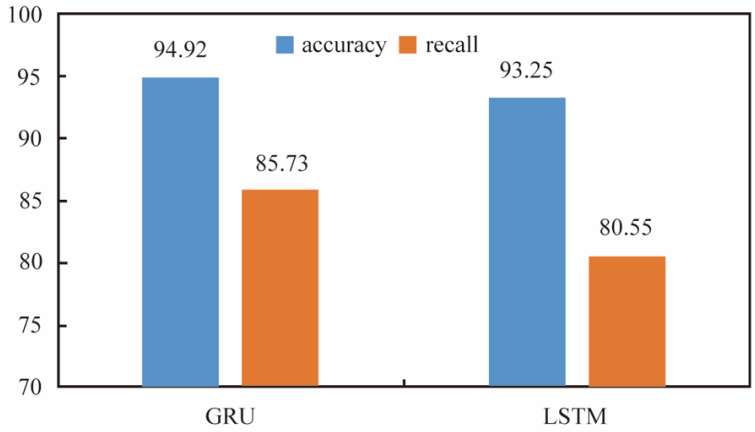

Figure 3. Failure prediction accuracy and recall of two time-series prediction models.

5.2.2. Container Migration Evaluation

We evaluated the container migration algorithm from two aspects: cost and degree of load balance. Figure 4 compares container migration costs under the three migration selection strategies. The cost is defined in Section 3.2. We fixed the total number of devices in the edge network at 30, and the number of devices predicted to fail gradually increased from 1 to 10. When the number of failures increases, the migration cost of the entire migration process correspondingly increases. This is because a suitable healthy node must be identified for the container in each failed device. Simultaneously, it can be clearly observed from the figure that the GA-based GACMS container migration strategy adopted in this study consumes less container migration cost than the GCMS and RCMS migration strategies.

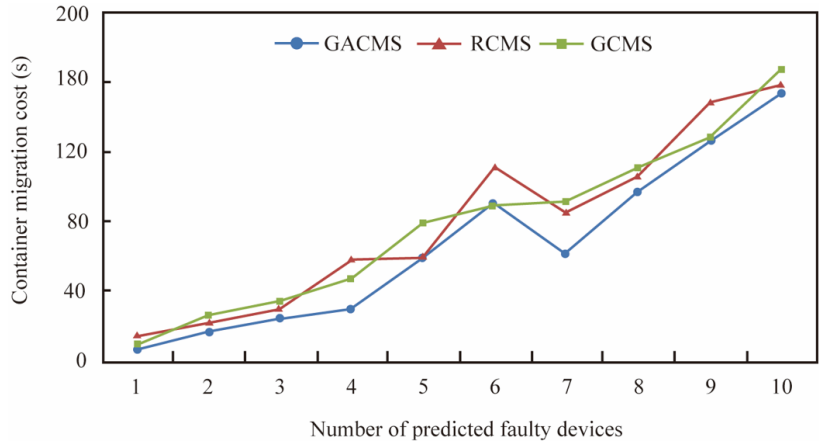

Figure 4. Container migration costs for different numbers of failed devices.

Figure 5 explores the load-balancing degree of the edge network when different numbers of edge devices fail in the edge network. While exploring the results of this experiment, we maintained a constant number of 30 devices in the edge network. The x-axis represents the number of predicted faulty devices, ranging from 1 to 10. The y-axis represents the load-balancing degree of the entire edge network after the container on the faulty device migrates to the surrounding healthy nodes. As shown in the figure, regardless of the number of faulty devices in the current edge network, GACMS exhibited the best degree of load balancing. Compared with the RCMS and GCMS, the proposed

GACMS can maximize the load balance of the entire edge network after container migration. Figures 4 and 5 show that GACMS outperforms the previous methods in terms of both migration cost and degree of load balancing.

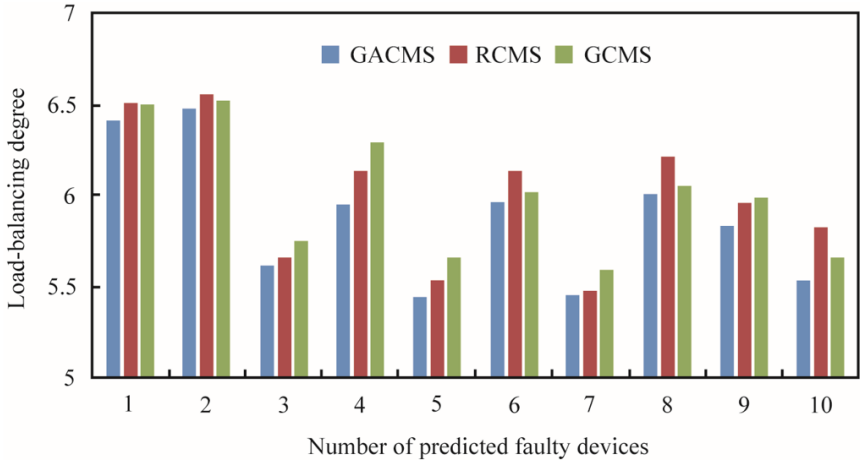

Figure 5. Degree of load balancing with respect to different numbers of edge-device failures.

Figure 6 shows the relationship between the total number of devices in the edge network and migration cost. In this experiment, we maintained the number of faulty devices at 10 and then changed the total number of devices in the edge network to 30, 40, and 50. The results show that the GACMS migration strategy adopted in this study has the lowest migration cost for the entire migration process compared with the RCMS and GCMS migration strategies. In other words, the proposed migration strategy under different scales of edge network equipment can achieve good results and effectively reduce the migration cost of container migration.

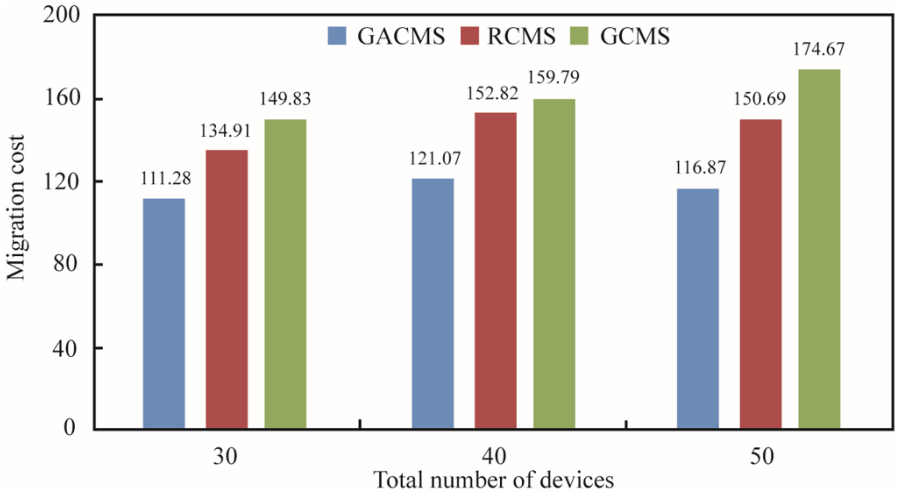

Figure 6. Migration costs for different numbers of edge network devices.

Figure 7 shows the relationship between the number of devices in the edge network and the degree of load balancing. When the number of devices in the edge network changes, the number of devices predicted to be faulty remains unchanged at 10. Therefore, as the total number of devices in the network increases, more resources become available in the edge network. When the value of the load-balance degree is lower, the load of the entire edge network is more balanced. The GACMS algorithm had a lower value for all three scales: 30, 40, and 50. This clearly shows that under different network scales, the container migration strategy based on the proposed GAMCS algorithm can better ensure the overall

load balance and migration cost of the edge network and, hence, can effectively improve the reliability of edge network services.

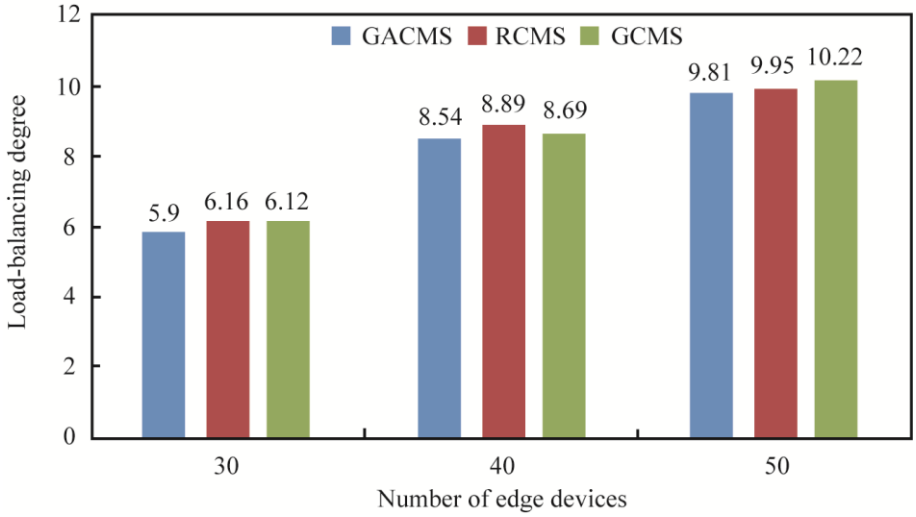

Figure 7. Degree of load balancing with respect to the size of the edge network.

6. Contributions and Future Work

In this paper, we propose a solution to service failure problems in an edge-computing environment. The contributions of this study are three-fold. First, we propose a novel two-phase method, the combination of fault prediction and container migration, on the edge computing. The fault prediction is used to decide when to trigger the container migration. The migration is to prevent the service failure. Second, this model can predict the possible downtime of equipment in advance through fault prediction. Third, we can model the container migration problem on the faulty device as an optimization problem and apply a reasonable container migration strategy in which containers on faulty devices can be migrated to other healthy devices to ensure service reliability. Our experiments show that this solution can overcome the service failure problem in edge-computing environments. The main research content of this study is summarized below.

Considering the prediction phase, it is difficult to predict the failure of edge devices. To achieve this, it is necessary to develop a fault-prediction algorithm that exhibits high accuracy. Therefore, we selected two commonly used time-series prediction algorithms: LSTM and GRU. By comparing the accuracy and recall rates of the two algorithms and the training loss function, we chose the GRU model with the best performance to predict the failure of edge devices. This approach achieves an accuracy of 0.94. Thus, our approach can significantly reduce false-positive container migration and hence reduce unnecessary waste of resources.

Considering the migration phase, if it is predicted that the device will fail, we must migrate the container deployed on the device. At present, container live migration technology can maintain the state of service, which can be used without the user's perception. However, if the migration is performed without considering the migration strategy, the number of containers migrating to a healthy device may be too large, and the device may be overloaded, which not only reduces the QoS but even causes the device to fail. Failure is the final downtime; therefore, a reasonable migration strategy must be considered. This study proposes a container migration strategy based on a GA that minimizes the migration cost and ensures the overall load balance of the network, thus effectively ensuring the service reliability of edge devices.

Finally, by combining the two methods of fault prediction and container migration, the service failure problem caused by the downtime of edge devices can be solved, the load balance of the edge network can be ensured, the migration cost of container migration can be reduced, and the entire edge network can achieve better reliability.

In the future, we would like to continue to explore this problem via the following two aspects:

(1) The fault-prediction algorithm used in this article requires historical fault data for support. However, in many cases, historical fault data may not exist. Therefore, in the future, we will develop a method that does not require historical fault data and characterize the normal status of a system via training with normal data. Then, we will perform fault prediction using the developed model.

(2) Regarding the migration strategy, it is easy to fall into local optimality when using GAs, and the training time is relatively long. In the future, we will adjust the GA to avoid this issue. We will also explore the use of other models to overcome this problem. Moreover, only load balancing and migration costs are currently considered, and additional factors must be considered considering the complexity of real-world scenarios.

Author Contributions: Conceptualization, L.L. and Z.Z.; methodology, Z.Z.; software, L.L.; validation, L.L. and Z.Z.; formal analysis, L.L.; investigation, L.K.; resources, Z.Z.; data curation, Z.Z.; writing—original draft preparation, L.L.; writing—review and editing, X.L.; visualization, Z.Z.; supervision, Z.Z.; project administration, Z.Z.; funding acquisition, Z.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the "Deep-time Digital Earth" Science and Technology Leading Talents Team Funds for the Central Universities for the Frontiers Science Center for Deeptime Digital Earth, China University of Geosciences (Beijing) (Fundamental Research Funds for the Central Universities; grant number: 2652023001), the National Natural Science Foundation of China under Grant 42050103 and 62372420, and China Geological Survey (CGS) work project: "Geoscience literature knowledge services and decision supporting." (project code: DD20230139).

Data Availability Statement: The availability of these data is subject to certain restrictions, as they have been utilized under license for the purpose of this study. Access to the data is contingent upon a review and approval process that involves a specific auditing procedure to ensure compliance with applicable regulations and ethical considerations. Researchers interested in accessing the data can initiate the request by contacting Lizhao LIU at 13908065625@163.com. Upon receipt of the request, further details regarding the review process and the necessary steps to obtain access will be provided. Please note that the availability of the data is dependent upon the successful completion of the review process.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Jalali, F.; Hinton, K.; Ayre, R.; Alpcan, T.; Tucker, R.S. Fog computing may help to save energy in cloud computing. *IEEE J. Sel. Areas Commun.* **2016**, *34*, 1728–1739. [CrossRef]
2. Hu, W.; Gao, Y.; Ha, K. Quantifying the impact of edge computing on mobile applications. In Proceedings of the 7th ACM SIGOPS Asia-Pacific Workshop on Systems, Hong Kong, China, 4–5 August 2016; pp. 1–8.
3. Zhao, L.; Liu, J. Optimal placement of virtual machines for supporting multiple applications in mobile edge networks. *IEEE Trans. Veh. Technol.* **2018**, *67*, 6533–6545. [CrossRef]
4. Wu, J.; Sheng, X.; Li, G.; Yu, K.; Liu, J. An efficient and secure aggregation encryption scheme in edge computing. *China Commun.* **2022**, *19*, 245–257. [CrossRef]
5. Lin, Q.W.; Hsieh, K.; Dang, Y.N. Predicting node failure in cloud service systems. In Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference (ESEC)/Symposium on the Foundations of Software Engineering (FSE), Athens, Greece, 23–28 August 2018; pp. 480–490.
6. Zhang, J.H.; Zhang, W.B.; Xu, J.W. Approach of virtual machine failure recovery based on hidden Markov model. *J. Softw.* **2014**, *25*, 2702–2714.
7. Di, Z.; Shao, E.; He, M. Reducing the time of live container migration in a workflow. In *IFIP International Conference on Network and Parallel Computing*; Springer: Cham, Switzerland, 2020; pp. 263–275.
8. Ma, Z.; Shao, S.; Guo, S.; Wang, Z.; Qi, F.; Xiong, A. Container migration mechanism for load balancing in edge network under power Internet of Things. *IEEE Access* **2020**, *8*, 118405–118416. [CrossRef]
9. Wang, T.; Gu, Z.; Zhang, W. Adaptive monitoring based fault detection for cloud computing systems. *Chin. J. Comput.* **2018**, *41*, 1112–1125.
10. Chen, M.Y.; Accardi, A.; Kiciman, E. Path-based failure and evolution management. In Proceedings of the 1st Symposiumon Networked Systems Design and Implementation, San Francisco, CA, USA, 29–31 March 2004; pp. 309–322.

11. Oppenheimer, D.; Ganapathi, A.; Patterson, D.A. Why do internet services fail, and what can be done about it? In Proceedings of the 4th Symposium on Internet Technologies and Systems, Seattle, WA, USA, 26–28 March 2003; pp. 1–16.
12. Dong, Z.; Ban, R.; Yang, Y. Research on the improved extreme learning machine algorithm in edge computing environment. In *Basic Clinical Pharmacology and Toxicology*; Wiley: Hoboken, NJ, USA, 2020; Volume 126, pp. 254–255.
13. Zhang, P.Y.; Shu, S.; Zhou, M.C. An online fault detection model and strategies based on SVM-grid in clouds. *IEEE CAA J. Autom. Sin.* **2018**, *5*, 445–456. [CrossRef]
14. Huang, H.; Ding, S.; Zhao, L. Real-time fault detection for IIoT facilities using GBRBM-based DNN. *IEEE Internet Things J.* **2019**, *7*, 5713–5722. [CrossRef]
15. Beloglazov, A.; Buyya, R. Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers. *Concurr. Comput. Pract. Exp.* **2012**, *24*, 1397–1420. [CrossRef]
16. Chou, L.D.; Chen, H.F.; Tseng, F.H.; Chao, H.; Chang, Y. DPRA: Dynamic power-saving resource allocation for cloud data center using particle swarm optimization. *IEEE Syst. J.* **2016**, *12*, 1554–1565. [CrossRef]
17. Paulraj, G.J.L.; Francis, S.A.J.; Peter, J.D.; Jebadurai, I.J. A combined forecast-based virtual machine migration in cloud data centers. *Comput. Electr. Eng.* **2018**, *69*, 287–300. [CrossRef]
18. Masoumzadeh, S.S.; Hlavacs, H. Dynamic virtual machine consolidation: A multi agent learning approach. In Proceedings of the 2015 IEEE International Conference on Autonomic Computing, Boston, MA, USA, 21–25 September 2015; IEEE Publications: Piscataway, NJ, USA, 2015; pp. 161–162.
19. Beloglazov, A.; Abawajy, J.; Buyya, R. Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing. *Future Gener. Comput. Syst.* **2012**, *28*, 755–768. [CrossRef]
20. Yin, L.; Li, P.; Luo, J. Smart contract service migration mechanism based on container in edge computing. *J. Parallel Distrib. Comput.* **2021**, *152*, 157–166. [CrossRef]
21. Omar, M.; Burrell, D. From text to threats: A language model approach to software vulnerability detection. *Int. J. Math. Comput. Eng.* **2023**, *2*, 23–24. [CrossRef]
22. Topaloğlu, G.; Kalaycı, T.A.; Pekel, K.; Akay, M.F. Revenue forecast models using hybrid intelligent methods. *Int. J. Math. Comput. Eng.* **2023**, *2*, 117–122. [CrossRef]
23. Wang, H.; Tianfield, H. Energy-aware dynamic virtual machine consolidation for cloud datacenters. *IEEE Access* **2018**, *6*, 15259–15273. [CrossRef]
24. Wen, Y.; Li, Z.; Jin, S.; Lin, C.; Liu, Z. Energy-efficient virtual resource dynamic integration method in cloud computing. *IEEE Access* **2017**, *5*, 12214–12223. [CrossRef]
25. Gao, Y.; Guan, H.; Qi, Z.; Hou, Y.; Liu, L. A multi-objective ant colony system algorithm for virtual machine placement in cloud computing. *J. Comput. Syst. Sci.* **2013**, *79*, 1230–1242. [CrossRef]
26. Li, K.; Chang, C.; Yun, K. Research on container migration mechanism of power edge computing on load balancing. In Proceedings of the 6th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA), Chengdu, China, 24–26 April 2021; IEEE Publications: Piscataway, NJ, USA, 2021; pp. 386–390.
27. Xin, C.; Yang, D.; Ma, Z. A load balancing container migration mechanism in the edge network. In *International Conference on Computer Engineering and Networks*; Springer: Singapore, 2020; pp. 1414–1422.
28. Ahmad, I.; Mehrotra, K.; Mohan, C.K.; Ranka, S.; Ghafoor, A. Performance modeling of load-balancing algorithms using neural networks. *Concurr. Pract. Exper* **1994**, *6*, 393–409. [CrossRef]
29. Yang, Y.; Dong, J.; Fang, C.; Xie, P.; An, N. FP-STE: A novel node failure prediction method based on spatiotemporal feature extraction in data centers. *Comput. Model. Eng. Sci.* **2020**, *123*, 1015–1031. [CrossRef]
30. Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.* **1997**, *9*, 1735–1780. [CrossRef] [PubMed]
31. Zhao, R.; Wang, D.; Yan, R. Machine health monitoring using local feature-based gated recurrent unit network. *IEEE Trans. Ind. Electron.* **2017**, *65*, 1539–1548. [CrossRef]
32. Chawla, N.V.; Bowyer, K.W.; Hall, L.O.; Kegelmeyer, W.P. SMOTE: Synthetic minority oversampling technique. *J. Artif. Intell. Res.* **2002**, *16*, 321–357. [CrossRef]